

Практична №1. Моделювання даних і вибір підходу.

Предметна область: система бронювання паркінгу.

Спочатку користувач реєструється в системі і потім може бронювати місця на себе. Система підтримує бронювання парковочного місця, скасування бронювання та пошук вільних місць.

Об'єкт	Факт	Сценарій
Користувач	Має ім'я, номер телефону та унікальний ідентифікатор	Користувач входить у систему
Паркомісце	Має номер, тип та статус	Система визначає вільне місце
Бронювання	Має номер заброньованого місця та ідентифікатор користувача, що забронював	Коли користувач натискає "забронювати" створюється бронювання з усіма даними

Сценарії

1. **"Бронювання"**. Створюється відповідний об'єкт, який зберігає номер заброньованого місця та ідентифікатор користувача. Також змінюється статус паркомісця з "вільне" на "заброньоване".
2. **"Реєстрація користувача"**. Створюється об'єкт користувача, якому відразу присвоюється унікальний ідентифікатор та ім'я і номер телефону користувача.

Підходи

1. *Ієрархічний*. Бронювання було би дочірнім для користувача.
2. *Мережевий*. Бронювання мало би зв'язки з об'єктом користувача та паркомісця.
3. *Реляційний*. Бронювання, користувачі та місця були б окремими записами в відповідних таблицях. Бронювання мало б номер місця та foreign key на користувача, у цій таблиці унікальним для кожної строки був би номер паркомісця, оскільки не можливо два рази забронювати одне й те саме місце.

4. *Об'єктний*. Кожна сутність була б об'єктом, а не записом, як в реляційній моделі.

5. *Документний*. Теж саме, що і реляційний підхід, але кожна сутність була би не записом, а файлом, наприклад формату json.

6. *Ключ-значення*. Значення будуть просто json об'єктами, як в документному підході. Але в бронюваннях замість номера паркомісця можна буде зберігати ключ на його сутність.

Висновок

Я б обрав реляційний підхід. Кожна сутність не потребує багато інформації про себе, тому зберігати їх я записи в таблицях буде найбільш компактно та зручно. Також це спрощує пошук, наприклад вільних місць.

Працювати з такими системами теж зручніше через їх більш просту організацію. Адміністрування в такому випадку теж простіше.